



# 114-2 NCKU Program Design-II HW5

## 繳交期限

2026年 06月 09日(Tue) 23:59:59

## 題目說明

你朋友是一個鄉下小快遞站的新員工，新來乍到的他對於這個工作充滿了熱情。

一直以來他們都是使用先到先送的原則來運送貨物，然而最近總公司推出了所謂"優先運送"的服務。

公司會根據客戶過去的使用紀錄、貨物的重量、額外購買特別服務等方面算出所謂的優先度。

根據優先度決定先運送那些貨物。

而你的朋友由於沒有合適的管理工具而總是排錯了運送表，這幾天對他們的客訴使得薪水快要扣光光了!!

因此這位朋友找到了你，希望你可以幫幫他。

## 題目規則

- 本作業有四個小題，每題25分
- 所有題目共用以下檔案
  - Cargo.h
  - Warehouse.h

## Q1 簡單排序

你的朋友剛接手理貨台，看著滿地散落的包裹滿頭大汗。目前他最需要的是一個最基礎的「登記與預覽」系統。

每天早上，貨車會載來一批包裹，他需要將這批包裹的資訊一次性輸入電腦中。

系統必須能自動把這些包裹按照「優先級」排好隊（優先級高的在前面；若優先級相同，則先輸入的排前面），並印出最終的預定派送清單，讓他可以直接印出來交給外務司機。

## 本題要求

撰寫Q1.cpp:

- 系統先讀取一個整數  $N$  ( $0 < N < 100$ )。
- 透過 `Cargo >>` 連續讀取  $N$  個包裹的資訊。
- 包裹的資訊為輸入一行string，內容為:地點 內容物 優先級  
(其中地點、內容物為全英文string，優先級為int)
- 使用 `Warehouse + Cargo` 將包裹放入倉庫並觸發排序。
- 使用 `Warehouse <<` 將排列好的派送序列印出。

## 輸入範例

Input:

```
5
Dawnstar Book 6
Rivendell Ring 8
Xanadu Arrow 3
Daggerfall Sword 7
Tamriel Arrow 2
```

Output:



Rivendell Ring 8  
Daggerfall Sword 7  
Dawnstar Book 6  
Xanadu Arrow 3  
Tamriel Arrow 2

## Q2 聯名特惠

雙11購物節到了！快遞公司與特定廠牌推出了「聯名特約」活動。

為了給客戶驚喜，所有與合作廠商有關的貨物都會註記為Collaboration，只要包裹的內容物是

**Collaboration**，公司將會免費將該包裹的優先級提升**1級**！

客戶是得到驚喜了，但你朋友嚇哭了。大量的包裹直接塞爆了你朋友的快遞站

你的朋友在忙亂中根本沒心思自己輸入時手動給特定商品升級。

他又找到了你，希望你能升級系統，讓系統在包裹入庫前，自動辨識並幫聯名包裹「加急」。

### 本題要求

撰寫Q2.cpp:

- 系統先讀取一個整數  $N$  ( $0 < N < 100$ )。
- 創建一個暫存的目標包裹，用來辨識後續輸入的商品
- 透過 `Cargo >>` 連續讀取  $N$  個包裹的資訊。
- 使用 `Cargo ==` 比對剛輸入的包裹內容物是否為Collaboration(聯名商品)。
- 使用Cargo的++運算子提升等級
- 最後使用 `Warehouse + Cargo` 入庫，並用 `Warehouse <<` 輸出派送單

### 輸入範例

Input:

5  
Tamriel Key 2  
Dawnstar Potion 4  
Dawnstar Amulet 3  
Karthwasten Collaboration 6  
Fargoth Ingot 6

Output:

Karthwasten Collaboration 7  
Fargoth Ingot 6  
Dawnstar Potion 4  
Dawnstar Amulet 3  
Tamriel Key 2

## Q3 多次出貨

一直以來因為經費的關係，快遞站只有一台車。

因此你朋友都是每天固定集貨到一個時間，再統一發送。

不過公司又給了一個"驚喜"，現在快遞站不缺車了。

因此現在隨時都有車子會來，也隨時都會有快遞員要出發投送。

你的朋友現在需要一個彈性更大的管理平台，讓他可以隨時輸入指令來應付包裹的進出。

### 本題要求

撰寫Q3.cpp:

實作一個 while 迴圈選單：

- 輸入 1：代表新貨到達。接著輸入  $N$  ( $0 < N < 100$ )，並使用 `Cargo >>` 與 `Warehouse + Cargo` 將  $N$  個包
- 輸入 2：代表司機出發。系統必須使用 `Warehouse --` 如果序列不為空，移除序列最前面的包裹（代表已出貨）。
- 輸入 0：代表今日營業結束。結束程式前，使用 `Warehouse <<` 印出今晚還留在倉庫中等待明天運送的包裹（數

### 輸入範例

Input:

```
1
3
Osgiliath Armor 10
Daggerfall Arrow 13
Valinor Ring 13
2
0
```

Output:

```
Valinor Ring 13
Osgiliath Armor 10
```

## Q4 排序加速

隨著業績蒸蒸日上，快遞站有時候一次會堆積好幾千件包裹。

朋友發現每次把新包裹放進去重新排序（`Warehouse + Cargo`）時，舊的演算法似乎讓電腦運作得非常緩慢，甚至有當機的風險。

因此你朋友又找到你，希望你可以給系統微調一下。

不過你也不確定新系統與舊系統差別有多少。

因此你決定修改完之後，跟原版的比對一下看哪個比較快，兩者間有什麼差距。

### 本題要求

撰寫 `Q4.cpp`:

- 複製一份 `Warehouse` 改名叫 `newWarehouse` 並修改其 `operator` 以實作不同的排序方法
- 在 `Q4.cpp` 中，利用課程中學到的 `time library` 的 `Function` 顯示兩者的工作時間差別
- 輸入使用 `while` 迴圈搭配 `>> operator`
- 最後應該分別輸出兩種方式的執行時間，以及它們間的差距

### 輸入範例

Input:

- 伺服器測試時，會先輸入 5000 筆資料，再分別對兩個 `Warehouse` 測試
- 你需要先接收所有輸入，因此建議使用下列程式讀取輸入

```
Cargo c;
while (cin >> c) {           // operator>> 讀取包裹
    TODO
}
```

- 本小題所使用的 `testcase` 有上傳至 moodle (`Q4testcase.txt`)

Output:



```
-----  
Warehouse takes: 6.257 seconds  
newWarehouse takes: 0.154 seconds  
-----  
newWarehouse is faster by 6.103 seconds!
```

## 評分測試

---

請學生將以下檔案上傳至同個資料夾

- Cargo.h
- Warehouse.h
- newWarehouse.h
- Q1.cpp
- Q2.cpp
- Q3.cpp
- Q4.cpp

學生請在上述檔案放置的資料夾中執行以下指令進行檢查

```
HW5
```

或是(中括號請省略)

```
HW5 [QuestionNumber] [TestCase]
```

請注意 最終評分請以執行 **HW5** 指令的結果為準

使用 `HW5 [QuestionNumber] [TestCase]` 的結果不代表最終分數